

ביצעתי 50 סיבובים עבור כל ניסוי אך מאחר והפלט ארוך הוספתי מכל ניסוי את הסיבוב הטוב ביותר ובקוד הוספתי את הניסוי שיצא הטוב ביותר מבין כל הניסויים.

1\15 experiences:

```
layers = [  
    tf.keras.layers.Flatten(input_shape=(28, 28)),  
    tf.keras.layers.Dense(53, activation='relu'),  
    tf.keras.layers.Dropout(0.2), # Dropout with 20% probability  
    tf.keras.layers.Dense(71, activation='relu'),  
    tf.keras.layers.Dropout(0.3), # Dropout with 30% probability  
    tf.keras.layers.Dense(45, activation='relu'),  
    tf.keras.layers.Dropout(0.4), # Dropout with 40% probability  
    tf.keras.layers.Dense(18, activation='relu'),  
    tf.keras.layers.Dropout(0.5), # Dropout with 50% probability  
    # Output Layer  
    tf.keras.layers.Dense(num_of_classes, activation='softmax'),  
]
```

Result-

Epoch 37/50

1875/1875 _____ **7s** 4ms/step - loss:
0.3388 - sparse_categorical_accuracy: 0.9043 - val_loss: 0.2732 -
val_sparse_categorical_accuracy: 0.9644

2\15 experiences:

```
layers = [  
    tf.keras.layers.Flatten(input_shape=(28, 28)),  
    tf.keras.layers.Dense(53, activation='relu'),  
    tf.keras.layers.Dense(71, activation='relu'),  
    tf.keras.layers.Dense(45, activation='relu'),  
    tf.keras.layers.Dense(18, activation='relu'),  
    # Output Layer  
    tf.keras.layers.Dense(num_of_classes, activation='softmax'),  
]
```

Result-

Epoch 18/50

1875/1875 _____ **11s** 4ms/step - loss:
0.0202 - sparse_categorical_accuracy: 0.9933 - val_loss: 0.1018 -
val_sparse_categorical_accuracy: 0.9796

3\15 experiences:

```
layers = [  
    tf.keras.layers.Flatten(input_shape=(28, 28)),  
    tf.keras.layers.Dense(53, activation='relu'),
```

```

tf.keras.layers.Dense(71, activation='relu'),
tf.keras.layers.Dense(45, activation='relu'),
tf.keras.layers.Dense(18, activation='sigmoid'),
# Output Layer
tf.keras.layers.Dense(num_of_classes, activation='softmax'),
]

```

Result-

Epoch 40/50

1875/1875 ————— **7s** 4ms/step - loss: 0.0105 - sparse_categorical_accuracy: 0.9972 - val_loss: 0.1183 - val_sparse_categorical_accuracy: 0.9778

4\15 experiences:

```

layers = [
    tf.keras.layers.Flatten(input_shape=(28, 28)),
    tf.keras.layers.Dense(53, activation='sigmoid'),
    tf.keras.layers.Dense(71, activation='sigmoid'),
    tf.keras.layers.Dense(45, activation='sigmoid'),
    tf.keras.layers.Dense(18, activation='sigmoid'),
    # Output Layer
    tf.keras.layers.Dense(num_of_classes, activation='softmax'),
]

```

Result-

Epoch 16/50

1875/1875 ————— **7s** 4ms/step - loss: 0.0382 - sparse_categorical_accuracy: 0.9899 - val_loss: 0.1192 - val_sparse_categorical_accuracy: 0.9708

5\15 experiences:

```

layers = [
    tf.keras.layers.Flatten(input_shape=(28, 28)),
    tf.keras.layers.Dense(53, activation='sigmoid'),
    tf.keras.layers.Dense(71, activation='relu'),
    tf.keras.layers.Dropout(0.2),
    tf.keras.layers.Dense(45, activation='sigmoid'),
    tf.keras.layers.Dense(18, activation='relu'),
    # Output Layer
    tf.keras.layers.Dense(num_of_classes, activation='softmax'),
]

```

Result-

Epoch 32/50

1875/1875 ————— **11s** 4ms/step - loss: 0.0218 - sparse_categorical_accuracy: 0.9930 - val_loss: 0.1247 - val_sparse_categorical_accuracy: 0.9764

6\15 experiences:

```
layers = [  
    tf.keras.layers.Flatten(input_shape=(28, 28)),  
    tf.keras.layers.Dense(53, activation='relu'),  
    tf.keras.layers.Dropout(0.2), # Dropout with 20% probability  
    tf.keras.layers.BatchNormalization(), #Batch Normalization  
    tf.keras.layers.Dense(71, activation='relu'),  
    tf.keras.layers.Dropout(0.2), # Dropout with 30% probability  
    tf.keras.layers.BatchNormalization(), #Batch Normalization  
    tf.keras.layers.Dense(45, activation='relu'),  
    tf.keras.layers.Dropout(0.2), # Dropout with 40% probability  
    tf.keras.layers.BatchNormalization(), #Batch Normalization  
    tf.keras.layers.Dense(18, activation='relu'),  
    tf.keras.layers.Dropout(0.2), # Dropout with 50% probability  
    tf.keras.layers.BatchNormalization(), #Batch Normalization  
    # Output Layer  
    tf.keras.layers.Dense(num_of_classes, activation='softmax'),  
]
```

Result-

Epoch 47/50

1875/1875 ————— **9s** 5ms/step - loss: 0.2060 - sparse_categorical_accuracy: 0.9471 - val_loss: 0.1010 - val_sparse_categorical_accuracy: 0.9740

7\15 experiences:

```
layers = [  
    tf.keras.layers.Flatten(input_shape=(28, 28)),  
    tf.keras.layers.Dense(53, activation='sigmoid'),  
    tf.keras.layers.Dropout(0.2), # Dropout with 20% probability  
    tf.keras.layers.BatchNormalization(), #Batch Normalization  
    tf.keras.layers.Dense(71, activation='sigmoid'),  
    tf.keras.layers.Dropout(0.2), # Dropout with 30% probability  
    tf.keras.layers.BatchNormalization(), #Batch Normalization  
    tf.keras.layers.Dense(45, activation='sigmoid'),  
    tf.keras.layers.Dropout(0.2), # Dropout with 40% probability
```

```

tf.keras.layers.BatchNormalization(), #Batch Normalization
tf.keras.layers.Dense(18, activation='sigmoid'),
tf.keras.layers.Dropout(0.2), # Dropout with 50% probability
tf.keras.layers.BatchNormalization(), #Batch Normalization
# Output Layer
tf.keras.layers.Dense(num_of_classes, activation='softmax'),
]

```

Result-

Epoch 44/50

1875/1875 ————— **9s** 5ms/step - loss: 0.1744 - sparse_categorical_accuracy: 0.9551 - val_loss: 0.0996 - val_sparse_categorical_accuracy: 0.9755

8\15 experiences:

```

layers = [
    tf.keras.layers.Flatten(input_shape=(28, 28)),
    tf.keras.layers.Dense(53, activation='sigmoid'),
    tf.keras.layers.BatchNormalization(), #Batch Normalization
    tf.keras.layers.Dense(71, activation='sigmoid'),
    tf.keras.layers.BatchNormalization(), #Batch Normalization
    tf.keras.layers.Dense(45, activation='sigmoid'),
    tf.keras.layers.BatchNormalization(), #Batch Normalization
    tf.keras.layers.Dense(18, activation='relu'),
    tf.keras.layers.BatchNormalization(), #Batch Normalization
    # Output Layer
    tf.keras.layers.Dense(num_of_classes, activation='softmax'),
]

```

Result-

Epoch 20/50

1875/1875 ————— **10s** 5ms/step - loss: 0.0364 - sparse_categorical_accuracy: 0.9881 - val_loss: 0.0788 - val_sparse_categorical_accuracy: 0.9791

9\15 experiences:

```

layers = [
    tf.keras.layers.Flatten(input_shape=(28, 28)),
    tf.keras.layers.Dense(53, activation='relu'),
    tf.keras.layers.BatchNormalization(), #Batch Normalization
    tf.keras.layers.Dense(71, activation='relu'),
    tf.keras.layers.BatchNormalization(), #Batch Normalization
]

```

```

tf.keras.layers.Dense(45, activation='relu'),
tf.keras.layers.BatchNormalization(), #Batch Normalization
tf.keras.layers.Dense(18, activation='relu'),
tf.keras.layers.BatchNormalization(), #Batch Normalization
# Output Layer
tf.keras.layers.Dense(num_of_classes, activation='softmax'),
]

```

Result-

Epoch 36/50

1875/1875 ————— **12s** 5ms/step - loss: 0.0282 - sparse_categorical_accuracy: 0.9906 - val_loss: 0.0757 - val_sparse_categorical_accuracy: 0.9809

10\15 experiences:

```

layers = [
    tf.keras.layers.Flatten(input_shape=(28, 28)),
    tf.keras.layers.Dense(53, activation= 'sigmoid',
kernel_regularizer=tf.keras.regularizers.l2(0.002)),
    tf.keras.layers.Dense(71, activation='sigmoid'),
    tf.keras.layers.BatchNormalization(), #Batch Normalization
    tf.keras.layers.Dense(45, activation='sigmoid'),
    tf.keras.layers.BatchNormalization(), #Batch Normalization
    tf.keras.layers.Dense(18, activation='sigmoid'),
    tf.keras.layers.BatchNormalization(), #Batch Normalization
    # Output Layer
    tf.keras.layers.Dense(num_of_classes, activation='softmax'),
]

```

Result-

Epoch 45/50

1875/1875 ————— **8s** 4ms/step - loss: 0.1388 - sparse_categorical_accuracy: 0.9722 - val_loss: 0.1467 - val_sparse_categorical_accuracy: 0.9704

11\15 experiences:

```

layers = [
    tf.keras.layers.Flatten(input_shape=(28, 28)),
    tf.keras.layers.Dense(53, activation='relu'),
    tf.keras.layers.Dense(71, activation='relu'),
    tf.keras.layers.Dense(45, activation='relu'),
    tf.keras.layers.Dense(18, activation='relu'),
]

```

```

tf.keras.layers.Dropout(0.4),
# Output Layer
tf.keras.layers.Dense(num_of_classes, activation='softmax'),
]

```

Result-

Epoch 36/50

1875/1875 _____ **10s** 3ms/step - loss: 0.0739 - sparse_categorical_accuracy: 0.9773 - val_loss: 0.2689 - val_sparse_categorical_accuracy: 0.9763

12\15 experiences:

```

layers = [
    tf.keras.layers.Flatten(input_shape=(28, 28)),
    tf.keras.layers.Dense(53, activation='sigmoid'),
    tf.keras.layers.Dense(71, activation='sigmoid'),
    tf.keras.layers.Dense(45, activation='sigmoid'),
    tf.keras.layers.Dense(18, activation='sigmoid'),
    tf.keras.layers.Dropout(0.4),
    # Output Layer
    tf.keras.layers.Dense(num_of_classes, activation='softmax'),
]

```

Result-

Epoch 48/50

1875/1875 _____ **9s** 3ms/step - loss: 0.0973 - sparse_categorical_accuracy: 0.9679 - val_loss: 0.2669 - val_sparse_categorical_accuracy: 0.9717

13\15 experiences:

```

layers = [
    tf.keras.layers.Flatten(input_shape=(28, 28)),
    tf.keras.layers.Dense(53, activation='relu'),
    tf.keras.layers.BatchNormalization(), #Batch Normalization
    tf.keras.layers.Dense(71, activation='relu'),
    tf.keras.layers.BatchNormalization(), #Batch Normalization
    tf.keras.layers.Dense(45, activation='relu'),
    tf.keras.layers.BatchNormalization(), #Batch Normalization
    tf.keras.layers.Dense(18, activation='relu'),
    tf.keras.layers.BatchNormalization(), #Batch Normalization
    tf.keras.layers.Dropout(0.2),
    # Output Layer
]

```

```
tf.keras.layers.Dense(num_of_classes, activation='softmax'),  
]
```

Result-

Epoch 41/50

1875/1875 ————— **11s** 5ms/step - loss: 0.0359 - sparse_categorical_accuracy: 0.9887 - val_loss: 0.0865 - val_sparse_categorical_accuracy: 0.9803

14\15 experiences:

```
layers = [  
    tf.keras.layers.Flatten(input_shape=(28, 28)),  
    tf.keras.layers.Dense(53, activation='relu'),  
    tf.keras.layers.BatchNormalization(), #Batch Normalization  
    tf.keras.layers.Dense(71, activation='relu'),  
    tf.keras.layers.Dense(45, activation='relu'),  
    tf.keras.layers.Dense(18, activation='relu'),  
    tf.keras.layers.Dropout(0.2),  
    # Output Layer  
    tf.keras.layers.Dense(num_of_classes, activation='softmax'),  
]
```

Result-

Epoch 49/50

1875/1875 ————— **10s** 3ms/step - loss: 0.0323 - sparse_categorical_accuracy: 0.9907 - val_loss: 0.1233 - val_sparse_categorical_accuracy: 0.9809

15\15 experiences:

```
layers = [  
    tf.keras.layers.Flatten(input_shape=(28, 28)),  
    tf.keras.layers.Dense(53, activation='sigmoid'),  
    tf.keras.layers.BatchNormalization(), #Batch Normalization  
    tf.keras.layers.Dropout(0.2),  
    tf.keras.layers.Dense(71, activation='sigmoid'),  
    tf.keras.layers.Dense(45, activation='sigmoid'),  
    tf.keras.layers.Dense(18, activation='sigmoid'),  
    # Output Layer  
    tf.keras.layers.Dense(num_of_classes, activation='softmax'),  
]
```

Result-

Epoch 21/50

1875/1875 **9s** 3ms/step - loss:
0.0882 - sparse_categorical_accuracy: 0.9731 - val_loss: 0.0862 -
val_sparse_categorical_accuracy: 0.9778